

Project 1 — SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint

Title: SupportDesk AI Chat And Auto Triage Any Customer Complaint

Type: Multi-agent (2 agents) · **Stack:** Python, raw OpenAI SDK (no extra library, on purpose) ·

Level: Beginner **Tagline:** A 2-agent support desk you run in your terminal — a Concierge agent that just chats, and a Triage agent that turns a free-text complaint into a clean, structured ticket.

Overview

SupportDesk AI is a terminal-based support tool built from two cooperating agents: a Concierge that holds a normal, remembering conversation, and a Triage agent that turns a messy, free-text complaint into a clean, structured ticket. A small router sits in front of both and automatically decides which one should handle each incoming message. It solves a real problem — a support inbox mixes casual questions with genuine complaints, and paying a person to sort every message by hand doesn't scale. Anyone building a first-version intake tool, support bot, or multi-purpose chatbot that needs to tell "just chatting" apart from "needs a ticket," without forcing the user to pick a category, would want something like this.

Features

- Real-time streaming chat agent (Concierge) that holds multi-turn conversations with real memory across turns
- Automatically classifies and extracts free-text complaints into a structured, validated `SupportTicket` (customer name, issue category, urgency, one-line summary)
- Automatic router that sends every incoming message to the correct agent — no manual mode-switching required
- Every routing decision is logged, so misrouted messages can be found and explained afterward
- Missing information (like a name never given) stays empty instead of being guessed or hallucinated
- Survives real-world failure cases without crashing: bad API key, rate limits, oversized conversations, malformed structured output

Tech Stack

- Python

- Raw OpenAI SDK (no agent framework, by design)
- Pydantic (structured output validation)
- python-dotenv (configuration and secrets)

Architecture

A small Python router inspects each incoming message and forwards it to one of two independent agents: the **Concierge** (a stateful chat agent that streams replies and remembers the growing conversation) or the **Triage** agent (a stateless extractor that turns free text into a validated `SupportTicket`). The two agents never talk to each other directly — the router is the only coordination point. This is the simplest possible multi-agent pattern: a sequential, routed handoff between two specialists, with no shared graph state and no tool-calling loop yet (those come in later projects).

Charter (what this project is)

A terminal app with two agents working together: Concierge (talks to the user) and Triage (turns a complaint into structured data). A simple router sends each message to the right one. No extra library yet — just the raw OpenAI SDK. This project proves two things with real code, not just reading: (1) you understand how to call an LLM correctly, and (2) you understand what "multi-agent" really means, at its simplest.

Jump to: [Setup](#) · [Step 1](#) · [Step 2](#) · [Step 3](#) · [Step 4](#)

The Story — what you're actually building

Imagine your company gets support messages all day — some people just want to chat or ask a quick question, others are genuinely upset about something broken and need a proper ticket filed. Right now a person reads every single message and decides which is which by hand. That's slow, and it doesn't scale.

You're building a small terminal app that does both jobs for you. One part, the **Concierge**, is a normal chat agent — it holds a real conversation, remembers what was said earlier, and replies naturally. The other part, the **Triage** agent, has a narrower, stricter job: read a messy, free-text complaint and turn it into a clean record a support system could actually use — who it's from, what kind of problem it is, how urgent it is, and a one-line summary. A small **router** sits in front of both and decides, automatically, which one should handle each message that comes in — so nobody has to say "this one's a complaint" out loud.

That's the whole finished product: type a message, and the app quietly decides whether you're chatting or filing a complaint, and responds accordingly. Everything underneath — a real API call, a

growing conversation, structured data extraction, and a router — comes together to make that one simple behavior work reliably.

Each Step below builds one piece of that, in order: Step 1 proves you can talk to the model at all. Step 2 turns that into a real, remembering conversation. Step 3 adds the Triage skill and the error-handling that keeps the whole thing from crashing on bad input. Step 4 adds the router that ties Concierge and Triage together into one automatic system — the actual "multi-agent" part.

Before you read further — think about it yourself: if you were building this by hand, what's the very first thing you'd need to prove works, before writing any chat logic at all? And once you have two different jobs (chatting, and filing a ticket), how would your code decide, on its own, which one a given message needs? Sit with these for a minute before you read the Steps below.

What you're actually building, in one line: a terminal app where a router function looks at each message and sends it to one of two agents — a chat agent, or a data-extraction agent.

Why this needs to exist: a real support inbox mixes casual questions with real complaints, and paying a person to sort every single message by hand does not scale as the number of messages grows.

When you'd reach for this at a real job: when a product starts getting more than one kind of incoming message, and nobody wants to force the user to pick a category themselves — a support inbox, a multi-purpose chatbot, any intake form.

How it works, mechanically: a small function reads the message and decides "concierge" or "triage," then calls the matching agent function — the two agents never talk to each other directly, the router just picks one per message.

Why not just do it some simpler/different way: why not write one big system prompt that handles both chatting and ticket extraction? Because one prompt trying to do two very different jobs (open-ended conversation, and strict structured output) tends to do both worse — the model gets confused about which mode it's in, and mixing a chatty tone with strict field-checking makes both harder to get right. Why not just always ask the user to pick a mode, like typing `/chat` or `/extract` ? Because that pushes the classification work onto the user instead of the system — the whole point of a real support tool is that it figures out what kind of message it got, on its own.

Where This Fits in the 5-Project Arc

Every project in this curriculum is a real multi-agent system. What changes from project to project is *how many* agents there are, *how* they're built, and *how* they work together. The number of agents grows as you go:

Project 1 (you are here)	→ 2 agents, simple Python router, raw SDK	(Con
Project 2	→ 2 agents, agent loop you build by hand, tools	(Wor
Project 3	→ 3 agents, built with LangGraph, RAG + human approva	
Project 4	→ 4-5 agents, one Supervisor directs the rest	(Sup
Project 5	→ same agents as Project 4, made production-ready (AP	

Each project adds one new skill on top of the last one — it's not just "more agents" for no reason. Project 1's 2 agents are kept simple on purpose (no tools, no extra library, a plain function decides which agent to use) so the only new idea is "a router sends work to the right agent." You're also learning the raw API at the same time (from Doc04), so this step doesn't add extra things to learn all at once.

Setup (do this once, before Step 1)

These are the very first commands to run. You don't need to read anything else first.

```
cd project_1_supportdesk
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install openai python-dotenv pydantic
pip freeze > requirements.txt
```

Create a file called `.env` in this folder (never commit this file to git):

```
OPENAI_API_KEY=sk-your-real-key-here
LOG_LEVEL=INFO
```

Create a second file, `.env.example`, with the same keys but no real values:

```
OPENAI_API_KEY=
LOG_LEVEL=INFO
```

Add `.venv/` and `.env` to your `.gitignore` file before your first commit.

Optional — real test complaints instead of writing your own: if you want realistic messy input for Step 3's Triage agent instead of making up your own test complaints, you can pull a few rows from the [CFPB Consumer Complaint Database](#) — a real, free, public dataset of consumer complaints (also mirrored on Kaggle, search "Consumer Complaint Database"). Not required — the 3 test inputs below work fine on their own.

A Real Example (so this isn't just theory)

The structured-output part of this project needs a real example to work with — not just "extract some fields." Use this one, or make your own, but keep it this specific:

Scenario: you're building a support-ticket extractor. The user describes a problem in plain text. Your code turns it into a `SupportTicket` :

```
customer_name: str | None
issue_category: Literal["billing", "technical", "account_access", "other"]
urgency: Literal["low", "medium", "high"]
summary: str          # one sentence describing the issue
```

Test these 3 inputs (save them in a `test_data.py` file so you reuse the same ones every time, instead of making up new ones each run): 1. "Hi, I'm Sarah Khan. I was charged twice for my subscription this month and I need it fixed today." → should give `issue_category="billing"`, `urgency="high"`, `customer_name="Sarah Khan"`. 2. "can't log into my account, tried resetting password twice, still nothing" → should give `issue_category="account_access"`, `customer_name=None` (no name was given — this checks that your code doesn't make up a name that isn't there). 3. "just wondering what your refund policy is, no rush" → `issue_category` could reasonably be "other" or "billing" — this one's a judgment call, and that's the point: decide, and be ready to explain your choice. `urgency="low"` .

Why input 2 matters: it checks that a missing piece of information stays empty (`None`) instead of the model guessing a name that was never mentioned. This is a real problem you'll actually hit, not a made-up one.

Why This Project Is Good for Your Portfolio

- **What problem it solves:** most simple "I called an LLM" demo projects don't handle a real back-and-forth conversation, don't produce clean structured data, and don't have more than one role talking to each other. This project does all three — and uses no extra library to do it.
- **Why it matters:** it shows you understand what's happening underneath the tools you'll use later. Someone looking at your code sees a clean 2-agent handoff built from scratch — not just an import from a library with no understanding behind it.
- **When you'd build something like this at a real job:** any small internal tool or first version of a product, where a full agent framework would be too much. A lot of real production features are exactly this: a small router in front of two or three well-defined model calls.
- **How it's built:** you type something in the terminal → a router decides Concierge or Triage → Concierge remembers the conversation and streams its reply, Triage does a one-time structured

extraction. Every likely failure (from Doc02/04) is handled on purpose, not left to crash the program.

Problems you'll likely run into, and how to fix them:

Problem	Fix
The conversation gets too long and goes over the model's limit	Count tokens as you add messages. Cut or shorten older messages before you hit the limit — don't wait for the API to reject the call
The structured-output call gives you something that doesn't match your format	Catch the Pydantic validation error by name. Try again once with a clearer instruction, or fail with a clear message — never pass bad data through
You find out your API key is wrong in the middle of a chat, not at the start	Test the key with one cheap call when the app starts. Fail fast, with a clear message, before the user types anything
The router picks the wrong agent on an ambiguous message	Log every routing decision (Step 4 item 4) so you can see what got misrouted, and why. If the same kind of wording keeps getting it wrong, switch from a keyword router to a model-call one, or add a third "ask a clarifying question" path instead of forcing a binary choice
Streaming breaks or hangs mid-reply	Wrap the streaming loop in a <code>try / except</code> for connection-level errors (like <code>APIConnectionError</code>). Always print a trailing newline in a <code>finally</code> block, so a dropped connection mid-stream doesn't leave the terminal stuck mid-line or the message list out of sync

Built During These Documents

[01_python_foundations](#) → [02_apis_http_json](#) → [03_llm_fundamentals](#) → [04_openai_api](#)

Plan Before You Code

See [15_five_projects_index](#): write down Problem → Requirements → Architecture → Components → Data Flow → Implementation Plan → Coding Tasks yourself, before you open your editor.

How To Build This — Step by Step

A note on this project's steps: the whole point of this project (see Doc04) is to learn the raw API *before* using any library. So unlike Projects 2-5, there's no "turn it into an agent" step here in the middle — Step 4 is where the multi-agent part appears, all at once, since the two "agents" are really just two functions with different jobs.

Step 1 — A Script That Sends One Message and Prints the Reply

Project: SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint — Step 1 of 4: A Script That Sends One Message and Prints the Reply

What this step does: proves the most basic thing works — your code can connect to the model and get back one correct reply. Nothing more. This step only checks "is my setup correct," separate from everything you'll add later. **When you'll hit this for real:** this exact "just prove the connection works" step is the first thing you should do on *any* new AI project, professional or personal — before building any feature, confirm the basic call works. **Read first:** [01_python_foundations Core Concepts](#) (config + logging), [04_openai_api Core Concepts — "The client and the message array"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Build `config.py` and `logging_setup.py` (or reuse them from `01_python_foundations` if you already built them there). No secrets typed directly into your code. 2. Write a script that sends **one** message to the model and prints the reply. No loop, no memory, no streaming yet. 3. Test it with something simple, like `"What 's 2+2?"`, and check the reply makes sense.

Your files after Step 1:

```
project_1_supportdesk_chat_and_triage/
├─ config.py           (or reused from Doc01)
├─ chat_client.py      → create_client(), send_message() only – no history yet
└─ main.py             → sends one fixed message, prints the reply
```

Step 2 — A Real Terminal Chat With Memory and Live Streaming

Project: SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint — Step 2 of 4: A Real Terminal Chat With Memory and Live Streaming

What this step does: turns your one-time call into a real conversation. **What's new vs. Step 1:** `send_message()` now takes in and returns a growing list of past messages, instead of sending just one message alone, and replies now stream in word by word instead of arriving all at once. **What stays the same:** `create_client()` and the basic API call — you're adding to Step 1's function, not throwing it away. **When you'll hit this for real:** every chat feature you will ever build starts with exactly this — a growing message list plus streaming. It's not specific to this project. **Read first:** [04_openai_api Core Concepts — "Streaming vs. waiting"](#), [03_llm_fundamentals Core Concepts — "Statelessness"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Change `send_message()` so it takes in and returns a growing `messages` list. This list **is** the memory — the model itself remembers nothing between calls (see Doc03), so your code

has to resend the whole history each time. 2. Put it in a loop: read what the user types, add it as a message, call the model, add the reply, repeat. 3. Add `stream_message()` and switch your loop to print each word as it arrives, instead of waiting for the full reply. 4. Test it: have a 3-message conversation where message 3 refers back to message 1 (like "what did I just ask you two messages ago?"). Check that the model actually remembers.

Your files after Step 2:

```
project_1_supportdesk_chat_and_triage/
├─ config.py
├─ chat_client.py      → create_client(), send_message(history, msg), stream_
└─ main.py             → a real chat loop with growing memory
```

Step 3 — A Triage Agent That Extracts a Structured Ticket From Free Text

Project: SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint — Step 3 of 4: A Triage Agent That Extracts a Structured Ticket From Free Text

What this step does: adds a second skill (turning text into structured data) next to your chat loop, and makes the whole app hard to crash. **What's new vs. Step 2:** a new `schemas.py` file, plus an "extract" mode; all 4 common failure cases now get handled properly. **What stays the same:** your Step 2 chat loop doesn't change at all — extraction is a separate path, not a change to how chat works. **When you'll hit this for real:** any feature where a user types free text and your system needs clean, structured data out of it — support tickets, form-filling, intake forms. This exact pattern. **Read first:** [04_openai_api Core Concepts — "Structured output" and "Error types"](#), [02_apis_http_json Core Concepts — "JSON: two different ways a response can be wrong"](#).

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write the `SupportTicket` Pydantic schema from the example above. This schema is the contract between the model's free-text reply and your code — get it right here, and Step 3's error handling becomes simple exception-catching instead of guesswork. 2. Add a way to trigger extraction mode instead of normal chat — for example, typing `/extract`, or a separate function. 3. Test it against all 3 example inputs from above. Check input 2's `customer_name=None` case specifically. 4. Handle all 4 common failures: bad API key (check it at startup, not mid-chat), rate limit (catch `RateLimitError`), too-long conversation (catch it, tell the user, don't crash), bad structured output (catch Pydantic's `ValidationError` specifically).

Your files after Step 3 (project's core is done):

```
project_1_supportdesk_chat_and_triage/
├─ config.py
├─ chat_client.py      → create_client(), send_message(), stream_message()
└─ schemas.py          → SupportTicket(BaseModel)
```

```

├─ test_data.py          → your 3 test inputs, reused every time
├─ main.py               → full chat loop + /extract mode + all error handling
└─ test_chat_client.py

```

Step 4 — A Router That Sends Each Message to the Right Agent Automatically (final: 2 agents)

Project: SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint — Step 4 of 4: A Router That Sends Each Message to the Right Agent Automatically

Read first: [11_multi_agent_systems Core Concepts — "Every pattern: WHAT/WHY/WHEN"](#) — look at the **Sequential** and **Supervisor** rows. You're building a tiny version of one of these, in plain Python, before you learn LangGraph in Doc11.

By the end of Step 3 you have two working pieces: a Concierge (Step 2's chat loop) and a Triage agent (Step 3's extractor). But you've been switching between them yourself, by typing `/extract`. Step 4 makes the *program* decide which one to use — and that's what actually makes this a multi-agent system, not just one script with two modes.

When you'll hit this for real: any system handling more than one kind of request, where a human shouldn't have to say which kind it is — a support inbox, a multi-purpose chatbot. This router pattern, small as it is here, is the same idea Project 4's Supervisor scales up.

Stuck on this step? [Hint 1](#) · [Hint 2](#) · [Show me the solution](#)

What to do: 1. Write a **Router**: a small, cheap function that decides Concierge or Triage for each message. This can be a simple keyword check, or a separate small model call with a very focused question, like "does this message describe a problem needing support — yes or no?" 2. Rename your two code paths as agents: `concierge_agent(history, message)` and `triage_agent(message) -> SupportTicket`. Give each one its own system prompt / personality. This is what actually makes them two separate agents, not just one function with an if-statement. 3. Connect the router to your main loop — the loop should no longer decide which path to run itself. It should ask the router, then send the message to whichever agent the router picks. 4. Log every routing decision: which message went to which agent, and why. This makes the whole thing checkable later, same idea as Project 2's trace log. 5. Test it: send a mix of normal chat and complaint-style messages in one session. Check that routing is correct for the clear cases, and actually look at (don't just accept) what happens on a tricky, unclear one.

Your files after Step 4 (final):

```

project_1_supportdesk_chat_and_triage/
├─ config.py
├─ chat_client.py
├─ schemas.py          → SupportTicket(BaseModel)
└─ test_data.py

```

```

├─ agents/
│   ├── router.py           → decide(message) -> "concierge" | "triage"
│   ├── concierge_agent.py  → concierge_agent(history, message) -> str
│   └── triage_agent.py     → triage_agent(message) -> SupportTicket
├─ main.py                 → loop: router.decide() → send to the right agent
└─ test_chat_client.py

```

Final Deliverable: SupportDesk-AI-Chat-And-Auto-Triage-Any-Customer-Complaint — a 2-agent terminal app. A Concierge agent holds real multi-turn conversations. A Triage agent turns any complaint into a checked, structured `SupportTicket`. A router decides which agent handles each message.

Why this really is multi-agent (said plainly): this is a simple **Sequential/routed** pattern (see Doc11). There's no tool-calling loop yet (that comes in Project 2), and no shared graph state yet (that comes in Project 3). But it genuinely is two agents with different jobs, and a router choosing between them — which is the real, simplest definition of multi-agent. Just don't describe it in your portfolio as more advanced than it is. Project 4 is where routing becomes a real `Command`-based supervisor.

Checklist Before You Call This Done

- [] Multi-turn conversation with real memory in one session (Concierge agent)
- [] Uses the config/logger from Doc01 — no secrets typed in code, no bare `print()`
- [] Structured-output feature tested against all 3 example inputs, including the case where a field is missing
- [] A router sends messages to the right agent automatically — not a manual `/extract` command
- [] Every routing decision is logged, with which agent handled it and why
- [] Survives: bad API key, rate limit, too-long conversation, bad structured output — without crashing
- [] You can explain every line of your own code, without help

Full requirements, test cases, and hints: [04_openai_api/README.md](#).

Status

Not started. Track your progress in [../PROGRESS.md](#).

Stuck? Ask for **Hint 1** or **Hint 2** about the exact part you're stuck on. Only ask for the full code if you say **"Show me the solution."**

Part of a 13-project multi-agent AI engineering curriculum. See the [full curriculum](#) for the complete learning path and all other projects.